

Lista 2 – INF05008

Instruções

- USE OS NOMES DE TIPOS DE DADOS E FUNÇÕES DEFINIDOS NAS QUESTÕES.
- Use o **template da solução disponível no Moodle**.
- TODAS as funções, inclusive as auxiliares, devem ter documentação completa: contrato, objetivo, e pelo menos 2 exemplos e 2 testes (só não precisa incluir testes nas funções que geram imagens). Se quiser, pode usar os testes como exemplos, mas neste caso isso deve ser indicado (leia o texto sobre exemplos e testes).
- Siga as instruções para a realização de exercícios disponível no Moodle (no item *Informações gerais*).
- Muitas vezes não é dito explicitamente no enunciado que um tipo deve ser definido, faz parte da solução identificar todos os tipos necessários para a realização dos exercícios.
- Para trabalhar com listas, você pode usar somente as funções pré-definidas **empty?**, **cons**, **list**, **length** e **append**. Não use outras funções pré-definidas sobre listas e nem funções de alta-ordem.
- O código cada função não pode ter mais que 8 linhas (sem contar linhas de comentários). Decomponha sua solução!
- Para todas as funções recursivas você **DEVE** incluir o modelo da solução. Esse modelo, **em forma de comentários que podem ser colocados antes ou permeados ao código**, deve explicitar como o algoritmo funciona e deve ter o seguinte formato:

Modelo de Algoritmo para Listas

Dados *uma lista L* e ...

Dizer quais as entradas do algoritmo.

se *< ...é o caso base da def. de lista... >*

Base: Identificar o caso trivial de listas e resolver o problema sem usar recursão.

então *< ...resolver o problema ... >*

se *< ... não é o caso base da def. de lista... >*

Passo: Pode haver mais de um passo, e pode-se usar senão, se for o último caso.

então *< ...combinar soluções... >*

< ...fazer algo com... > < o primeiro elemento da lista >
< ...solucionar o problema para... > < o resto da lista >

Atenção: No modelo da solução evite usar a palavra *recursão* (ou palavras derivadas desta): quando você sentir necessidade de dizer *e aplica a função recursivamente ao resto da lista*, diga o que essa aplicação deve devolver (por exemplo *a soma dos elementos do resto da lista*, *a imagem dos elementos do resto da lista*, *o menor dos números do resto da lista*, ...) e descreva como, a partir desse resultado (da aplicação recursiva), é construído o resultado da função. Note que dependendo do que o algoritmo deve fazer, pode ou não ser necessário combinar as soluções, usar o primeiro elemento da lista ou mesmo o resto da lista. O modelo acima deve ser adaptado em cada exercício. Exemplo (lembre que o leitor deve conseguir entender como o algoritmo funciona lendo este modelo, pois ele é a descrição da solução):

TAMANHO

Objetivo: Determinar o tamanho de uma lista.

Dada *uma lista L*

se *a lista L estiver vazia*

então *devolver zero*

senão *somar um ao TAMANHO do resto da lista L*

Nos exercícios a seguir vamos trabalhar com listas. A linguagem de ensino de Racket oferece diferentes representações sintáticas para listas, nós vamos usar duas delas nesta disciplina:

Representando listas usando cons e empty

Uma lista é uma estrutura recursiva, obtida através da inserção de vários elementos, um a um, em uma lista vazia. Portanto, a representação de uma lista pode ser obtida através de uma operação que insere um elemento em uma lista qualquer, e um valor que representa uma lista vazia. Na linguagem Racket, utilizamos a função `cons` para inserir um elemento em uma lista, e o valor `empty` representa a lista vazia. Assim, uma lista com os elementos 23, 4 e 2, nesta ordem, é representada por

```
(cons 23 (cons 4 (cons 2 empty)))
```

onde

(`cons 2 empty`) representa a lista obtida inserindo o número 2 na lista vazia (`empty`);

(`cons 4 (cons 2 empty)`) representa a lista obtida inserindo o número 4 na frente da lista que tem apenas o número 2;

(`cons 23 (cons 4 (cons 2 empty))`) representa a lista obtida inserindo o número 23 na frente da lista que já tem um 4 e um 2.

Essa representação pode não parecer tão adequada do ponto de vista da legibilidade, mas ela enfatiza a natureza recursiva das listas, cada lista carrega consigo a forma como ela foi construída (que determina, também, a forma na qual ela deve ser processada por programas que manipulam listas).

Note que listas podem ter elementos repetidos.

Representando listas usando list

Muitas vezes nós (humanos) preferimos uma representação mais abstrata e compacta de listas. Em Racket, podemos usar o operador `list` para representar listas^a. Por exemplo, a lista com os elementos 23, 4 e 2, nesta ordem, é representada por

```
(list 23 4 2)
```

Mas note que, internamente, essa lista é traduzida para (`cons 23 (cons 4 (cons 2 empty))`), ou seja, apesar de visualizarmos a lista de uma forma mais compacta, ela continua sendo uma estrutura definida recursivamente usando `cons` e `empty`.

^aSe você quiser usar essa representação, deve selecionar a linguagem *Beginning Student with List Abbreviations* no DrRacket.

Nos exercícios, você pode usar qualquer representação para listas, mas lembre sempre que `list` é uma abstração muito útil para construir exemplos. Na maioria dos programas recursivos é utilizada a função `cons`. Você pode usar as funções pré-definidas `length`, que devolve o tamanho de uma lista, e a função `append`, que concatena duas (ou mais) listas. **Selecione a linguagem *Beginning Student with List Abbreviations* no DrRacket.**

O Tangram é um quebra-cabeças geométrico chinês formado por 7 peças, chamadas tans: são 2 triângulos grandes, 2 pequenos, 1 médio, 1 quadrado e 1 paralelogramo. Utilizando todas essas peças sem sobrepor-las, podemos formar várias figuras. (fonte: Wikipedia)

As peças podem ser todas da mesma cor ou de cores diferentes. A imagem abaixo é um exemplo das peças do Tangram. O modelo disponibilizado para esta lista de exercícios já contém constantes para desenhar cada uma destas peças, bem como a definição do tipo de dado `PeçaTangram`.



Não esqueça de colocar o modelo da solução para as funções recursivas – será descontada nota de quem não colocar!

1. Defina o tipo de dados chamado `ListaDePeças` para armazenar listas de peças do Tangram. Defina pelo menos 5 constantes do tipo `ListaDePeças`, sendo que ao menos duas listas de peças devem ter elementos repetidos.
2. Desenvolva a função `conta-peças-tipo` que, dadas uma peça e uma lista de peça, devolve o número de vezes que esta peça aparece na lista.

Como as peças são imagens, para compará-las você deve usar a função `image=?`.

3. Construa a função `há-repetidas?` que, dada uma lista de peças, verifica se há peças repetidas nesta lista.
4. Construa a função `peças-todos-tipos?` que, dada uma lista de peças, verifica se há peças de todos os 7 tipos de peças Tangram na lista. Como as peças tem cores diferentes, para ter todas as peças a lista deve ter as peças das duas cores diferentes, no caso dos triângulos grandes e pequenos.
5. Um professor realizou uma atividade com Tangram com seus alunos. Para isso, ele disponibilizou vários kits de Tangram e os alunos deveriam montar figuras com as peças. Ele avisou aos alunos que cada um deveria usar todas as 7 peças de um kit apenas em suas figuras. Porém, os alunos nem sempre seguiram a especificação, então houve figuras que não usaram todas as peças e figuras com peças repetidas. Defina um tipo de dados chamado `Figura` para armazenar as figuras dos alunos. Cada figura deve conter o nome do aluno, a lista de peças utilizadas e o conceito (a atribuição de conceitos será definida na próxima questão). Defina também o tipo de dados `ListaDeFiguras`, que contém as figuras definidas pelos alunos de uma turma. Note que nesta definição estamos abstraindo qual figura cada aluno desenhou, apenas está sendo armazenada a informação sobre quais peças foram usadas. Defina pelo menos 4 constantes do tipo `Figura` e 2 do tipo `ListaDeFiguras`.

6. Como descrito na questão anterior, os conceitos das figuras dos alunos serão dados considerando as peças utilizadas:

conceito A : se há peças de todos os 7 tipos diferentes e não há peças repetidas;

conceito B : se há peças de todos os tipos, mas há alguma repetida;

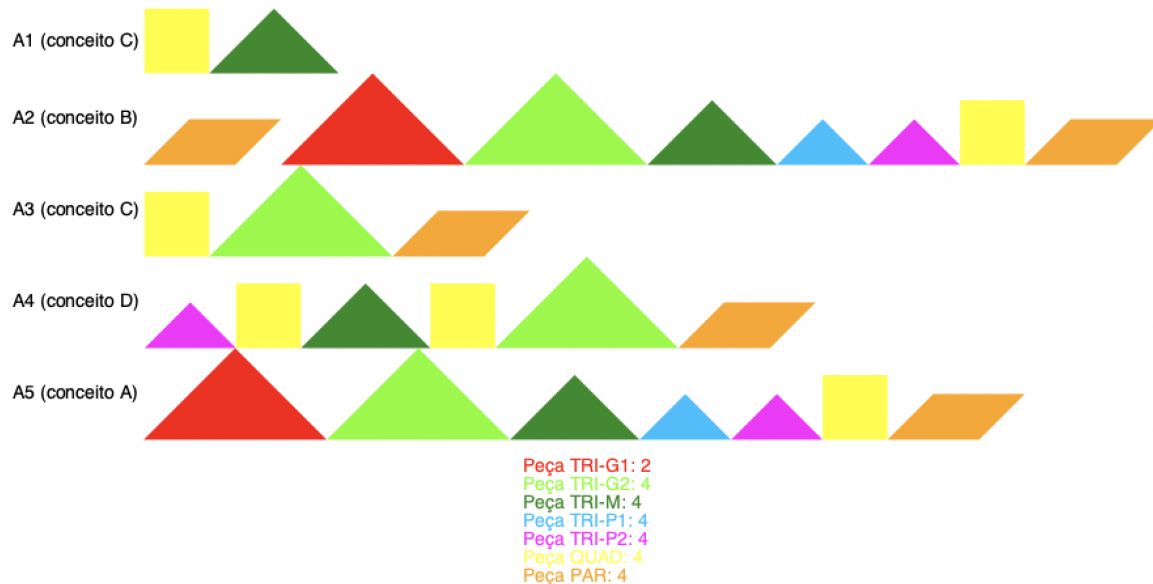
conceito C : se não há repetidas, mas faltou algum tipo de peça (e a figura do aluno contém peças);

conceito D : caso nenhum dos casos acima se aplique.

Construa a função que realiza esta avaliação, definindo o conceito de uma figura. A função deve se chamar `atribui-conceito` e o resultado deve ser o registro da figura com o conceito correspondente à sua lista de peças no campo do conceito da figura. Faça, também, a função `atribui-conceitos` que, dada uma lista de figuras, atribui conceitos para todas as figuras da lista, gerando uma lista de figuras com os conceitos.

7. Construa a função **análise-das-figuras** que, dada uma lista de figuras de uma turma, gera uma imagem mostrando os nomes dos alunos, seus conceitos, as peças utilizadas em suas figuras, e mostrando também o número total de peças de cada tipo que foram usadas nas figuras da turma. Você pode escolher a melhor forma de mostrar os dados de cada figura, bem como os totais. A imagem abaixo é um exemplo, você pode fazer de outra forma.

Dica: Decomponha o problema em problemas menores, resolva separadamente e combine as soluções. Sempre que um problema for difícil demais para resolver, imagine que existe uma outra função resolve parte do problema e tente construir a solução como se esta função existisse. Depois, é só implementar essa outra função (pode ser necessário quebrar essa função em outras mais simples também). Sempre que você notas que está escrevendo o mesmo código novamente, pense se não seria melhor construir uma função para isso e chamá-la em vários lugares. A melhor forma de resolver um problema normalmente não é através da construção de uma função complexa, e sim da combinação de várias funções simples!



8. Faça um desenho usando as peças Tangram, e dê o nome de **MINHA-FIGURA** para seu desenho.

*Para fazer desenhos é útil usar as funções **rotate**, para girar as peças, e **overlay/xy**, para posicionar peças uma relação à outra, considerando deslocamentos nos eixos x e y .*